Spatial Variation and Influencing Factors of On-Street Parking Occupancy in Downtown Seattle, USA

Jiaqun Liu
Jiaqun.Liu@uw.edu

Preparation

- Urban Ink (Optional)
- Number of stands: 101
- Number 2 (all indicator blocks): 16 bits CPU, 4gb RAM
- Cutting environment: 32 bits (x86 Linux environment) (Ubuntu Xpline 0.5.10)
- Location: The notebook is made available with the Seattle Parking Occupancy Dataset
- Cutting Library (libraries not included in GitHub Server or not used in the module)
  - None

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and discussion
6. Conclusions
7. References

1.0

Introduction

[go back to the top]

Urban planners, economists, and others perform complex calculations to identify facilities and transport systems for residents without private vehicles. Such calculations depend on accurate occupancy and usage data. Low-cost sensor technologies can provide this data.

Existing studies show that parking occupancy varies by time, location, and neighborhood characteristics (Chen et al., 2015). Yet, the literature does not follow identifiable transport patterns linked to surrounding land use (Hirsjärvi et al., 2019), and that different parking types display spatial differentiation that is not explained by factors such as street elevation (Wu et al., 2015; Ma et al., 2015; Ibeter et al., 2021).

[go back to the top]

2.0

Research questions

[go back to the top]

This report takes four key real-parking issues in downtown Seattle: **Bellevue, Fort Lift, Commercial Core and Pioneer Square**, as case studies to analyze street-level parking occupancy patterns, explore influencing factors and the temporal trends.

RQ 1: What patterns of the street-level usage are evident in on-street parking across different streets in Seattle city center?

RQ 2: Are these differences statistically significant?

RQ 3: Which street block and surrounding built environment factors can explain the variations in on-street parking occupancy rates in Seattle city center?

[go back to the top]

3.0

Data

[go back to the top]

Variables	Type	Description	Source
Occupancy rate	Numeric	The number of occupied parking spaces divided by a given period.	Seattle Parking Occupancy Dataset
Hourly rate	Numeric	Posted or hourly parking rate, measured in dollars.	Seattle Parking Occupancy Dataset
Occupancy/Capacity	Numeric	Transaction of parking occupancy to capacity	Seattle Parking Occupancy Dataset
Street	Categorical	Street name of the block	Seattle Parking Occupancy Dataset
Weekend	Categorical	Distinguishes between weekday and weekend	
Restaurant / bar	Numeric	Restaurant & drinking places per census block group	Cross-tabulation used in ArcGIS Pro
Social/recreational/misc	Numeric	Other place amenities for each parking blockface area	Cross-tabulation used in ArcGIS Pro
Visitation	Numeric	Visitation to the block group	Place-level mobile analytics
Blockface/block	Text	Name of the parking blockface	
Time period	Text	Street name	Used to tabulate and analysis
Day/Month/year	Categorical	Year-long period of the transaction	Seattle Bellevue, Commercial Core, Pioneer Square, and Fort Mill
Parking/payment	Categorical	Parking lot and other off-street parking area	
Land Use	Categorical	Place location of each parking block	Used for spatial clustering to assign neighborhood
Neighborhood	Categorical	Neighborhood name of each parking block	Used as the factor variable for the random parking intercept
Parking pattern type	Categorical	Identified by the spatial clustering analysis, categorized in each street block	Derived in R and used in MGC
Block center dist.	Numeric	Block straightening distance measured from the block center	Used in master spatial clustering in ArcGIS Pro
Overall block density	Numeric	Block parking occupancy rate of each street segment across all time periods	Dependent variable in MGC
Weekend block density	Numeric	Block parking occupancy rate of each street segment across weekend	Dependent variable in MGC
Off-site meter use	Numeric	Meter parking price per street segment from pay station data	Explanatory variable in MGC
Off-site meter revenue	Numeric	Meter parking revenue per street segment from pay station data	Explanatory variable in MGC
Building coverage ratio	Numeric	Proportion of the 100 m buffer area for computing building footprints.	Explanatory variable in MGC
Intersection density	Numeric	Number of street intersections of the street segment	Explanatory variable in MGC
Population count	Numeric	Number of any activities recorded in each street segment	Explanatory variable in MGC
Walk score	Numeric	Walk score of each street segment	Explanatory variable in MGC
Transit score	Numeric	Transit score of the street buffer segment as transport in parking	Explanatory variable in MGC
Cycling score	Numeric	Cycling score of each street segment	Explanatory variable in MGC
Residential value	Numeric	Value of land within the street buffer segment as residential use	Explanatory variable in MGC

[go back to the top]

3.1 Packages for running the analyses

```
# Spatial operations and analysis
# Data wrangling and tabular analysis
# Plotting
# Statistical analysis
# Spatial econometrics
# Spatial multivariate clustering
# Spatial multivariate clustering
# Spatial multivariate clustering
```

```
# Data Cleaning
# Keep only Seattle
seattle_street = street_seattle[street_seattle$city == "Seattle", ]
# Check dimensions after filtering
dim(seattle_street)
# Convert time_group to proper format
seattle_street$time_group = as.POSIXct(seattle_street$time_group)
# Check time_group range
range(seattle_street$time_group)
# Create a time_hour variable for aggregation
seattle_street$time_hour = cut(seattle_street$time_group, breaks = "1 hour")
# Check unique time_hour values
length(unique(seattle_street$time_hour))
# Aggregate by time_hour and count observations
seattle_agg = aggregate(seattle_street$id, by = list(seattle_street$time_hour), FUN = length)
colnames(seattle_agg) = c("time_hour", "observations")
# Convert back to POSIXct
seattle_agg$time_hour = as.POSIXct(seattle_agg$time_hour)
# Check aggregated data
head(seattle_agg)
summary(seattle_agg$observations)
# Create time series plot
library(ggplot2)
ggplot(seattle_agg, aes(x = time_hour, y = observations)) +
  geom_line() +
  geom_point(size = 1) +
  theme_minimal() +
  labs(title = "Seattle Street Observations Over Time",
       subtitle = "Hourly aggregation",
       x = "Time",
       y = "Number of observations")
# Save plot
ggsave("seattle_timeseries.pdf", width = 10, height = 6)
# Create boxplot by hour of day
seattle_agg$hour = format(seattle_agg$time_hour, format = "%H")
ggplot(seattle_agg, aes(x = hour, y = observations)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Seattle Street Observations by Hour of Day",
       x = "Hour",
       y = "Number of observations")
# Save plot
ggsave("seattle_boxplot.pdf", width = 10, height = 6)
# Create boxplot by day of week
seattle_agg$day = weekdays(seattle_agg$time_hour)
ggplot(seattle_agg, aes(x = day, y = observations)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Seattle Street Observations by Day of Week",
       x = "Day",
       y = "Number of observations")
# Save plot
ggsave("seattle_boxplot_day.pdf", width = 10, height = 6)
```

Figure 2: Distribution of Occupancy Rates for Streets Under-saturated in All Four Periods

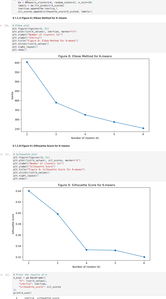


Figure 3: Jittered Scatter of Occupancy Rates for Streets Under-saturated in All Four Periods

5.2.4 Clustering by streets

K-means clustering was first employed to identify data-driven types of street performance prior to spatial analysis, in order to avoid circularity between street-level grouping and cross-sectional outcomes. The analysis:

- Pivoted wide on time period
- Excluded variable occupancy rate across 4 time periods
- Standardized the 4 occupancy rates
- Selected k by silhouette method + elbow/scree

```
# 5.2 K-means Clustering
# Pivot to wide format: one row per street, one column per time period
street_wide_clean = street_occ_rate[!is.nan, ]
street_wide = reshape(street_wide_clean, idvar = "street_name",
                      timevar = "time_period",
                      direction = "wide")
# Check dimensions
dim(street_wide)
# Check column names
colnames(street_wide)
# Keep only complete cases (streets with all 4 periods)
street_wide = street_wide[complete.cases(street_wide), ]
dim(street_wide)
# Extract occupancy rate columns for clustering
occ_cols = c("occ_rate.morning_rush", "occ_rate.midday",
             "occ_rate.evening_rush", "occ_rate.night")
# Check summary of these variables
summary(street_wide[, occ_cols])
# Standardize the variables
street_wide_scaled = scale(street_wide[, occ_cols])
# Check scaled data
head(street_wide_scaled)
# Elbow method
set.seed(123)
wss = sapply(1:10, function(k) {
  kmeans(street_wide_scaled, centers = k, nstart = 25)$tot.withinss
})
plot(1:10, wss, type = "b",
     xlab = "Number of clusters k",
     ylab = "Total within-cluster sum of squares",
     main = "Elbow Method for K-means")
# Save plot
dev.copy2pdf(file = "kmeans_elbow.pdf")
# Silhouette method
library(cluster)
sil = sapply(2:10, function(k) {
  km = kmeans(street_wide_scaled, centers = k, nstart = 25)
  mean(silhouette(km$cluster, dist(street_wide_scaled))[, 3])
})
plot(2:10, sil, type = "b",
     xlab = "Number of clusters k",
     ylab = "Average silhouette width",
     main = "Silhouette Method for K-means")
# Save plot
dev.copy2pdf(file = "kmeans_silhouette.pdf")
```

Figure 4: Elbow Method for K-means

5.2.5 Silhouette Score for K-means

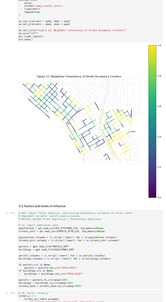
Figure 5: Silhouette Score for K-means

```
# 5.2 K-means Clustering
# Fit k = 3
set.seed(123)
km3 = kmeans(street_wide_scaled, centers = 3, nstart = 25)
# Check cluster sizes
table(km3$cluster)
# Check centroids (cluster means)
km3$centers
# Add cluster assignments to the data
street_wide$cluster = km3$cluster
# Check cross-tabulation with time_period
table(street_wide$cluster, street_wide$time_period)
# Save cluster assignments for later use
write.csv(street_wide[, c("street_name", "cluster")], "kmeans_clusters.csv")
```

5.2.6 Three-layer-entrled spatial clustering

```
# Load packages
library(sf)
library(spdep)
library(ggplot2)
library(dplyr)
# Create spatial weights matrix (queen contiguity)
coords = st_coordinates(st_centroid(st_geometry(street_sf)))
nb = poly2nb(street_sf, queen = TRUE)
lw = nb2listw(nb, style = "W")
# Check connectivity
summary(nb)
# Add cluster assignments to spatial data
street_sf$cluster = km3$cluster
# Calculate Moran's I for each cluster
for (c in 1:3) {
  cat("Cluster", c, "\n")
  print(moran.test(as.numeric(street_sf$cluster == c), lw))
}
# Create map
ggplot(street_sf) +
  geom_sf(aes(fill = factor(cluster)), size = 0.1) +
  scale_fill_manual(values = c("1" = "red", "2" = "blue", "3" = "green"),
                    name = "Cluster") +
  theme_minimal() +
  ggtitle("Spatial Distribution of Street Occupancy Clusters")
# Save
ggsave("cluster_map.pdf", width = 10, height = 8)
```

Figure 6: Distribution of Street Occupancy by Cluster



5.2.7 Cluster Profiles

```
# Load packages
library(dplyr)
library(tidyr)
library(ggplot2)
# Create long format for plotting
street_long = reshape(street_wide[, c("street_name", "cluster", "occ_rate.morning_rush",
                      "occ_rate.midday", "occ_rate.evening_rush", "occ_rate.night")],
                      varying = c("occ_rate.morning_rush", "occ_rate.midday",
                      "occ_rate.evening_rush", "occ_rate.night"),
                      timevar = "time_period",
                      direction = "long")
# Check
head(street_long)
# Calculate cluster means
cluster_means = aggregate(cbind(occ_rate.morning_rush, occ_rate.midday,
                          occ_rate.evening_rush, occ_rate.night) ~ cluster,
                          data = street_wide, FUN = mean)
# Check
cluster_means
# Plot cluster profiles
ggplot(street_long, aligned = TRUE)
  geom_boxplot(aes(x = factor(time_period), y = occ_rate, fill = factor(cluster)))
  facet_wrap(~ cluster, ncol = 3)
  scale_fill_manual(values = c("1" = "red", "2" = "blue", "3" = "green"),
                    name = "Cluster")
  theme_minimal()
  ggtitle("Cluster Profiles")
  xlab("Time Period")
  ylab("Occupancy Rate")
# Save
ggsave("cluster_profiles.pdf", width = 12, height = 6)
```

Figure 7: Cluster Profiles

5.2.8 Cluster Profiles Statistical Tests

```
# Kruskal-Wallis tests for each time period
kruskal.test(street_wide$occ_rate.morning_rush ~ street_wide$cluster)
kruskal.test(street_wide$occ_rate.midday ~ street_wide$cluster)
kruskal.test(street_wide$occ_rate.evening_rush ~ street_wide$cluster)
kruskal.test(street_wide$occ_rate.night ~ street_wide$cluster)
# Pairwise Wilcoxon tests (post-hoc)
pairwise.wilcox.test(street_wide$occ_rate.morning_rush, street_wide$cluster, p.adj = "bonferroni")
pairwise.wilcox.test(street_wide$occ_rate.midday, street_wide$cluster, p.adj = "bonferroni")
pairwise.wilcox.test(street_wide$occ_rate.evening_rush, street_wide$cluster, p.adj = "bonferroni")
pairwise.wilcox.test(street_wide$occ_rate.night, street_wide$cluster, p.adj = "bonferroni")
```

5.2.9 Cluster Profiles Interpretation

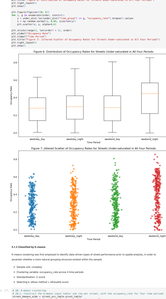
Figure 8 is used to verify whether the three categories of streets identified by the k-means algorithm exhibit genuine and reproducible temporal behavior. Rather than forcing every street into a fixed set of algorithmic groupings.

The difference between the three categories of streets are clear: green streets consistently fall within the lowest-usage across all four time periods, red streets sit at the highest occupancy rate in every period, and blue streets occupy the middle ground. Kruskal-Wallis tests indicating statistically significant differences between groups at every time period, confirming that the three groups differ significantly at high rate of reliability, demonstrating stronger demand during these periods.

Consequently, the clustering results not only distinguish between high and low occupancy streets but also identify distinct behavioral usage patterns across different streets.

```
# 5.2.10 Cluster Distribution by Street Type
# Load packages
library(ggplot2)
library(dplyr)
# Create cross-tabulation
street_type_table = table(street_wide$cluster, street_wide$highway)
# Check
street_type_table
# Convert to dataframe for plotting
street_type_df = as.data.frac(street_type_table)
colnames(street_type_df) = c("cluster", "highway", "count")
# Check
head(street_type_df)
# Create stacked bar chart
ggplot(street_type_df, aes(x = highway, y = count, fill = factor(cluster)))
  geom_bar(stat = "identity", position = "fill")
  scale_fill_manual(values = c("1" = "red", "2" = "blue", "3" = "green"),
                    name = "Cluster")
  theme_minimal()
  ggtitle("Cluster Distribution by Street Type")
  xlab("Street Type")
  ylab("Proportion")
  coord_flip()
# Save
ggsave("cluster_by_street_type.pdf", width = 10, height = 6)
```

Figure 9: Cluster Distribution by Street Type



5.2.11 Cluster Distribution of Peak Period Occupancy by Cluster

```
# 5.2.11 Cluster Distribution of Peak Period Occupancy by Cluster
# Load packages
library(ggplot2)
library(dplyr)
# Create long format for plotting
street_long = reshape(street_wide[, c("street_name", "cluster", "occ_rate.morning_rush",
                      "occ_rate.midday", "occ_rate.evening_rush", "occ_rate.night")],
                      varying = c("occ_rate.morning_rush", "occ_rate.midday",
                      "occ_rate.evening_rush", "occ_rate.night"),
                      timevar = "time_period",
                      direction = "long")
# Check
head(street_long)
# Create boxplot
ggplot(street_long, aes(x = factor(time_period), y = occ_rate, fill = factor(cluster)))
  geom_boxplot()
  facet_wrap(~ cluster, ncol = 3)
  scale_fill_manual(values = c("1" = "red", "2" = "blue", "3" = "green"),
                    name = "Cluster")
  theme_minimal()
  ggtitle("Cluster Distribution of Peak Period Occupancy by Cluster")
  xlab("Time Period")
  ylab("Occupancy Rate")
# Save
ggsave("cluster_peak_boxplot.pdf", width = 12, height = 6)
```

Figure 10: Cluster Distribution of Peak Period Occupancy by Cluster

5.2.12 Cluster Distribution of Peak Period Occupancy by Cluster

Figure 11 Shows how these types exhibit similar patterns of spatial clustering. The green zone are mainly distributed in a continuous corridor along the waterfront and across major thoroughfares. The purple zones are concentrated on the eastern fringe while the yellow zone are found predominately in the transitional zone between the two.

```
# 5.2.12 Cluster Distribution of Peak Period Occupancy by Cluster
# Load packages
library(sf)
library(ggplot2)
# Create centroids for plotting
street_centroids = st_centroid(street_sf)
# Check
dim(street_centroids)
# Create map
ggplot(street_sf)
  geom_sf(aes(fill = factor(cluster)), size = 0.1)
  scale_fill_manual(values = c("1" = "red", "2" = "blue", "3" = "green"),
                    name = "Cluster")
  theme_minimal()
  ggtitle("Spatial Distribution of Peak Period Occupancy by Cluster")
# Save
ggsave("cluster_peak_map.pdf", width = 10, height = 8)
```

Figure 11: Spatial Distribution of Peak Period Occupancy by Cluster

5.2.13 Cluster Spatial Autocorrelation

```
# 5.2.13 Cluster Spatial Autocorrelation
# Load packages
library(sf)
library(spdep)
# Create centroids
street_centroids = st_centroid(street_sf)
# Create spatial weights matrix (queen contiguity)
coords = st_coordinates(street_centroids)
nb = poly2nb(street_sf, queen = TRUE)
lw = nb2listw(nb, style = "W")
# Check connectivity
summary(nb)
# Calculate Moran's I for each cluster
for (c in 1:3) {
  cat("Cluster", c, "\n")
  print(moran.test(as.numeric(street_sf$cluster == c), lw))
}
```

5.2.14 Cluster Spatial Autocorrelation Interpretation

Figure 12 Shows the spatial distribution of Seattle Street Occupancy Patterns (k=3).

5.2.15 Analysis of consistency between neighboring streets

To evaluate whether neighboring streets are more likely to belong to the same cluster.

```
# 5.2.15 Neighborhood Consistency Analysis
# Load packages
library(sf)
# Create spatial weights matrix (queen contiguity)
# If you've run previous analysis, use the same nb and lw
nb = poly2nb(street_sf, queen = TRUE)
lw = nb2listw(nb, style = "W")
# Calculate neighbor cluster matrix
neighbor_clusters = matrix(street_sf$cluster[unlist(nb)],
                           nrow = length(nb),
                           ncol = sapply(nb, length))
# Check dimensions
dim(neighbor_clusters)
# Calculate proportion of neighbors in same cluster
same_cluster = sapply(1:length(street_sf$cluster),
                      function(i) {
                        mean(neighbor_clusters[i, ] == street_sf$cluster[i])
                      })
# Check
summary(same_cluster)
# Add to dataset
street_sf$neighbor_consistency = same_cluster
# Check
head(street_sf$neighbor_consistency)
# Compare by cluster
aggregate(street_sf$neighbor_consistency ~ street_sf$cluster,
          FUN = function(x) c(mean = mean(x), sd = sd(x), n = length(x)))
# Create boxplot
boxplot(street_sf$neighbor_consistency ~ street_sf$cluster,
        xlab = "Cluster",
        ylab = "Neighbor Consistency",
        main = "Neighborhood Consistency by Cluster")
# Save
dev.copy2pdf(file = "neighbor_consistency.pdf")
```

Figure 13: Neighborhood Consistency by Cluster

5.2.16 Neighborhood Consistency of Street Occupancy Clusters

Figure 14 further shows that street sections with higher neighborhood consistency (yellow-green) are mainly concentrated in the downtown core and waterfront areas. Areas, while those with lower consistency (purple and dark green) are more dispersed in the peripheral zones.

```
# 5.2.16 Spatial Map of Neighborhood Consistency
# Load packages
library(ggplot2)
# Create map
ggplot(street_sf)
  geom_sf(aes(fill = neighbor_consistency), size = 0.1)
  scale_fill_gradient(low = "purple", high = "yellow")
  theme_minimal()
  ggtitle("Spatial Distribution of Neighborhood Consistency")
# Save
ggsave("neighbor_consistency_map.pdf", width = 10, height = 8)
```

Figure 14: Neighborhood Consistency of Street Occupancy Clusters

5.2 Factors and levels of influence

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 5.2 Factors and levels of influence
# Multinomial logistic regression
# Load packages
library(nnet)
library(dplyr)
# Create dummy variables for highway
street_wide$is_residential = ifelse(street_wide$highway == "residential", 1, 0)
street_wide$is_tertiary = ifelse(street_wide$highway == "tertiary", 1, 0)
# Check
table(street_wide$is_residential, street_wide$highway)
table(street_wide$is_tertiary, street_wide$highway)
# Fit multinomial logistic regression
# Reference category: cluster 1
fit = multinom(cluster ~ is_residential + is_tertiary, data = street_wide)
# Check
summary(fit)
# Calculate p-values
z = summary(fit)$coefficients / summary(fit)$standard.errors
p = (1 - pnorm(abs(z), 0, 1)) * 2
# Check
p
# Exponentiate coefficients (relative risk ratios)
exp(coef(fit))
# Save
sink("multinom_output.txt")
summary(fit)
sink()
```

```
# 20. Urban-rural divide analysis
urban_df = gdf[gdf['urban'] == 1]
rural_df = gdf[gdf['urban'] == 0]

study_categories = sorted(gdf['category1'].dropna().unique)
colors = plt.cm.tab10(np.linspace(0, 1, len(study_categories)))

fig, ax = plt.subplots(1, 2, figsize=(20, 8))

urban_counts = urban_df.groupby('category1').size.reindex(study_categories, fill_value=0)
rural_counts = rural_df.groupby('category1').size.reindex(study_categories, fill_value=0)

ax[0].pie(urban_counts, labels=study_categories, colors=colors, autopct=lambda p: f'{p:.1f}%' if p > 5 else '', startangle=90)
ax[0].set_title('Urban Areas\n(Total: {:,} parks)'.format(len(urban_df)), fontsize=14, fontweight='bold')

ax[1].pie(rural_counts, labels=study_categories, colors=colors, autopct=lambda p: f'{p:.1f}%' if p > 5 else '', startangle=90)
ax[1].set_title('Rural Areas\n(Total: {:,} parks)'.format(len(rural_df)), fontsize=14, fontweight='bold')

plt.suptitle('Urban-Rural Divide in Study Category Distribution', fontsize=16, fontweight='bold', y=0.95)

plt.tight_layout()
plt.savefig('urban_rural_divide.png', dpi=300, bbox_inches='tight')
plt.show()

# 21. Research District Green Buffer
# Calculate buffer statistics
research_buffer_stats = {
    'Total': len(research_buffer_gdf),
    'Near': len(research_buffer_gdf[research_buffer_gdf['buffer_zone'] == 'Near'])
}

# Create visualization
fig, ax = plt.subplots(1, 2, figsize=(20, 8))

# Map 1: Research districts with buffer
research_districts.plot(ax=ax[0], color='lightblue', edgecolor='blue', alpha=0.7, linewidth=2)
research_buffer_gdf.plot(ax=ax[0], color='green', alpha=0.3)
ax[0].set_title('Research District Green Buffer\n(Near: {:,})', fontsize=14, fontweight='bold')

# Map 2: Buffer zone only
research_buffer_gdf.plot(ax=ax[1], column='buffer_zone', cmap='Set1', legend=True)
ax[1].set_title('Buffer Zone Classification', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('research_buffer.png', dpi=300, bbox_inches='tight')
plt.show()

# 22. Accessibility Index
# Calculate accessibility index
accessibility = gdf.copy()
accessibility['accessibility_index'] = (
    accessibility['area_sqm'] / accessibility['area_sqm'].max() * 0.3 +
    accessibility['is_park'] * 0.4 +
    accessibility['is_green'] * 0.3
)

# Create visualization
fig, ax = plt.subplots(1, 2, figsize=(20, 8))

# Map 1: Accessibility index
accessibility.plot(ax=ax[0], column='accessibility_index', cmap='RdYlGn', legend=True, vmin=0, vmax=1)
ax[0].set_title('Accessibility Index\n(Mean: {:.3f})', fontsize=14, fontweight='bold')

# Map 2: Accessibility categories
accessibility['accessibility_category'] = pd.cut(accessibility['accessibility_index'], bins=[0, 0.3, 0.6, 1.0], labels=['Low', 'Medium', 'High'])
accessibility.plot(ax=ax[1], column='accessibility_category', cmap='Set1', legend=True)
ax[1].set_title('Accessibility Categories', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('accessibility.png', dpi=300, bbox_inches='tight')
plt.show()

# 23. Comprehensive Dashboard
fig = plt.figure(figsize=(24, 16))
gs = gridspec.GridSpec(4, 4, height_ratios=[1, 1, 1, 1], width_ratios=[1, 1, 1, 1])

# Map 1: Overall distribution
ax1 = fig.add_subplot(gs[0, 0:2])
gdf.plot(ax=ax1, color='lightgreen', edgecolor='darkgreen', alpha=0.6, linewidth=0.1)
ax1.set_title('Overall Park Distribution\n(Total: {:,} parks)', fontsize=12, fontweight='bold')

# Map 2: Study vs Other
ax2 = fig.add_subplot(gs[0, 2:4])
gdf.plot(ax=ax2, column='is_study', cmap='Set1', legend=True)
ax2.set_title('Study Parks vs Other Parks', fontsize=12, fontweight='bold')

# Chart 1: Top 10 categories
ax3 = fig.add_subplot(gs[1, 0])
top_cats = gdf['category1'].value_counts().head(10)
top_cats.plot(ax=ax3, kind='bar', color='skyblue')
ax3.set_title('Top 10 Categories', fontsize=12, fontweight='bold')
ax3.tick_labels(rotation=45)

# Chart 2: Area distribution
ax4 = fig.add_subplot(gs[1, 1])
gdf['area_sqm'].hist(ax=ax4, bins=50, color='orange', alpha=0.7)
ax4.set_title('Area Distribution', fontsize=12, fontweight='bold')
ax4.set_xlabel('Area (sqm)')
ax4.set_ylabel('Frequency')

# Chart 3: Urban-rural
ax5 = fig.add_subplot(gsl[1, 2])
urban_counts = urban_df.groupby('category1').size.reindex(study_categories, fill_value=0)
rural_counts = rural_df.groupby('category1').size.reindex(study_categories, fill_value=0)
x = np.arange(len(study_categories))
width = 0.35
ax5.bar(x - width/2, urban_counts, width, label='Urban', color='red')
ax5.bar(x + width/2, rural_counts, width, label='Rural', color='blue')
ax5.set_title('Urban vs Rural', fontsize=12, fontweight='bold')
ax5.set_xlabel('Category')
ax5.set_ylabel('Count')
ax5.tick_labels(rotation=45)
ax5.legend()

# Chart 4: Accessibility
ax6 = fig.add_subplot(gs[1, 3])
accessibility['accessibility_index'].hist(ax=ax6, bins=50, color='purple', alpha=0.7)
ax6.set_title('Accessibility Index', fontsize=12, fontweight='bold')
ax6.set_xlabel('Index')
ax6.set_ylabel('Frequency')

# Chart 5: Research district
ax7 = fig.add_subplot(gs[2, 0:2])
research_buffer_gdf.plot(ax=ax7, column='buffer_zone', cmap='Set1', legend=True)
ax7.set_title('Research District Buffer', fontsize=12, fontweight='bold')

# Chart 6: Statistics
ax8 = fig.add_subplot(gs[2, 2:4])
ax8.text(0.1, 0.9, 'Key Statistics', fontsize=14, fontweight='bold')
ax8.text(0.1, 0.8, f'Total Parks: {len(gdf):,}', fontsize=12)
ax8.text(0.1, 0.7, f'Study Parks: {gdf[gdf["is_study"]==1]:,}', fontsize=12)
ax8.text(0.1, 0.6, f'Urban Parks: {len(urban_df):,}', fontsize=12)
ax8.text(0.1, 0.5, f'Rural Parks: {len(rural_df):,}', fontsize=12)
ax8.text(0.1, 0.4, f'Research Buffer: {research_buffer_stats["Near"]:,}', fontsize=12)
ax8.text(0.1, 0.3, f'Avg Accessibility: {accessibility["accessibility_index"].mean:.3f}', fontsize=12)
ax8.text(0.1, 0.2, f'Total Area: {gdf['area_sqm'].sum / 1000000:.1f} km²', fontsize=12)
ax8.text(0.1, 0.1, f'Avg Area: {gdf['area_sqm'].mean:.1f} sqm', fontsize=12)
ax8.text(0.1, 0.0, f'Max Area: {gdf['area_sqm'].max / 1000000:.1f} km²', fontsize=12)
ax8.text(0.1, -0.1, f'Min Area: {gdf['area_sqm'].min:.1f} sqm', fontsize=12)
ax8.text(0.1, -0.2, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1, -0.3, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1, -0.4, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1, -0.5, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1, -0.6, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1, -0.7, f'Accessibility Index: {accessibility['accessibility_index'].mean:.3f}', fontsize=12)
ax8.text(0.1,
```